

Project Report:

Extending the LiSA Static Analyzer

Sommario

1. Introduction & Analysis Details.....	3
1.1. Analyzer Configuration & Core Design Choices	3
Context-Sensitive Interprocedural Analysis	3
1.2. Mathematical Foundations	3
1.3. Mathematical Abstract Domains	4
1.3.1. Constant Propagation Domain	4
1.3.2. Sign Domain	4
1.3.3. Parity Domain	5
1.3.4. Interval Domain.....	5
1.3.5. Taint Three Level Domain.....	5
2. Widening Operator ()	7
2.1. Ascending Chain Condition (ACC)	7
2.2. Narrowing Operator ()	7
3. Abstract Domains	9
3.1. Extended Sign Domain (ExtendedSign & ExtendedSignLattice)	9
3.1.1. ExtendedSignLattice	9
3.1.2. ExtendedSign	9
3.2. Float Interval Domain (FloatInterval & FloatIntervalLattice)	10
3.2.1. FloatInterval.....	10
3.2.2. FloatIntervalLattice.....	10
3.3. Sign-Taint Combined Domain (SignTaint & SignTaintLattice).....	11
3.3.1. SignTaintLattice	11
3.3.2. SignTaint	12
4. Semantic Checkers	13
4.1. Architectural Integration and State Extraction	13
4.2. Lower Bound Validation (Underflow Prevention)	14
4.3. Upper Bound Validation and Heap Traversal	14
4.4. The Relational Advantage of the Pentagon Domain	15
4.5. Experimental Validation and Results	15
4.5.1. Test Methodology	15
4.5.2. Results Discussion	17

5.	Results on the Custom Tests	20
5.1.	Division by zero	20
5.2.	Numerical Overflow and Underflow	20
5.3.	String Prefix and Suffix	21
5.4.	Three-Level Taint	21
6.	Flow Graphs and Abstract State Analysis	22
6.1.	Numerical Analysis: Loop Convergence and Widening	22
6.2.	Combined Domain Analysis: Joint Extended Sign and Taint Propagation	23
7.	Cross-Analysis of Peer Programs	25
7.1.	Div-by-zero: 880119-890558-divbyzero.imp	25
7.2.	Overflow: 894004_overflow.imp	25
7.3.	HTTP Prefix Domain (872966_Strings.imp & 891184_strings.imp)	26
7.4.	Dot-com suffix: 880119-890558-dotcom-strings.imp	26
7.5.	Three-level taint: 876957-taint.imp	26
8.	Key Findings and Technical Insights	27
8.1.	Relational Domains are Mandatory for Array Loops	27
8.2.	Floating-Point Termination Requires Sacrificing Precision	27
8.3.	Context Sensitivity is Non-Negotiable for Security	27
8.4.	The Branch Refinement Gap	27
8.5.	The "Multiply by Zero" Annihilator is Lost in Abstraction	28
8.6.	Widening Erases Loop Bounds in Overflow Analysis	28
9.	Conclusions and Future Work	29
9.1.	Expanding Relational Analysis (The Pentagon Domain)	29
9.2.	Optimizing the Cartesian Product (Reduced Product & Refinement)	29
9.3.	Advanced Program Verification and Security Checkers	30
9.4.	Final Remarks: Bridging Theory and Practice	30

1. Introduction & Analysis Details

This report describes the engineering and extension of the LiSA (*Library for Static Analysis*) framework to perform semantic code analysis on programs written in the IMP language. The primary goal of this project is to leverage the theoretical principles of **Abstract Interpretation** to automatically identify software bugs and security vulnerabilities.

The development of this project followed two core paths:

- **Custom Abstract Domains:** Designing and implementing fine-grained abstract domains (Extended Sign, SignTaint, and Float Interval) to model execution properties.
- **Semantic Checkers:** Constructing static program verifiers that intercept control flow graph (CFG) elements to detect vulnerabilities such as division by zero, numerical overflows, data leakage, and out-of-bounds memory accesses.

1.1. Analyzer Configuration & Core Design Choices

Statically analyzing software requires balancing precision against computation time. To achieve high precision while mathematically proving the termination of our analysis, we configured LiSA with advanced interprocedural mechanisms and custom fixpoint operators.

Context-Sensitive Interprocedural Analysis

The analyzer is configured to run an interprocedural analysis using LiSA's `ContextBasedAnalysis` engine. In static analysis, a context-insensitive approach merges dataflow properties from all distinct call sites of a function using a least upper bound (*lub*) operator, which introduces significant precision loss and false positives.

By using context-sensitive analysis, the framework generates separate abstract states for each distinct calling context. This choice is vital for our semantic checkers. For example, when checking string operations or mathematical helpers, the engine analyzes utility methods like `download(url)` independently for each call site. The context-specific analysis correctly propagates the precise parameters from the caller down to the sink, allowing the checker to flag a vulnerability only where an actual dangerous input is passed, completely avoiding false alarms on safe execution paths.

1.2. Mathematical Foundations

To rigorously formalize how properties are tracked and approximated within the LiSA framework, the analyzer relies on the mathematical structures defined by **Lattice Theory**. Abstract domains are structurally organized as **Complete Lattices**, which provide the algebraic guarantees required to compute sound approximations and ensure fixpoint convergence.

- **Partially Ordered Sets (Posets):** An abstract domain is fundamentally a poset $(D^\#, \sqsubseteq)$, where $D^\#$ represents the set of abstract properties and $\sqsubseteq$ is the partial order relation. This relation mathematically models the concept of *relative precision* or *relative information content*: if $a \sqsubseteq b$, then a is more precise (contains more information) than b , meaning the concrete values represented by a are a subset of those represented by b ($\gamma(a) \subseteq \gamma(b)$). The relation $\sqsubseteq$ is strictly reflexive ($a \sqsubseteq a$), antisymmetric ($a \sqsubseteq b \wedge b \sqsubseteq a \Rightarrow a = b$), and transitive ($a \sqsubseteq b \wedge b \sqsubseteq c \Rightarrow a \sqsubseteq c$).
- **Least Upper Bound (LUB / Join / $\sqcup$):** Given a subset of abstract states, the Least Upper Bound represents the unique smallest element in the lattice that is greater than or equal to all

components in that subset. In static analysis, the LUB operator ($\sqcup$) is used at **control-flow merge points** (e.g., where the then and else branches of a conditional statement recombine). The LUB computes a safe, conservative over-approximation that encompasses all possible concrete execution paths, deliberately trading off precision to preserve mathematical soundness.

- **Greatest Lower Bound (GLB / Meet / $\sqcap$):** Conversely, the Greatest Lower Bound represents the unique largest element that is less than or equal to all elements in a given subset. The GLB operator ($\sqcap$) mathematically models **information refinement** and state filtering. It is heavily utilized during the evaluation of conditional expressions or loop guards (e.g., under an assume statement), where the analyzer combines the current abstract state with the constraint imposed by the guard to restrict the range of possible values.
- **Top ($\top$) and Bottom ($\perp$):** In Abstract Interpretation, a domain is a mathematical lattice $(\mathcal{L}, \leq)$ that maps concrete program values (like integers or strings) to abstract representations. A Complete Lattice always features two extreme global bounds:
 - **Bottom ($\perp$):** The least element of the lattice, representing the complete absence of concrete behaviours or an **unreachable program state** (dead code).
 - **Top ($\top$):** The greatest element of the lattice, representing **maximum uncertainty** or the complete lack of static information (i.e., the variable could hold any value, making it impossible for the analyzer to rule out runtime errors).

By structuring our custom domains (Extended Sign, Float Intervals, and SignTaint) as complete lattices adhering to these properties, we ensure that every semantic evaluation possesses a mathematically defined upper and lower bound, paving the way for predictable and sound fixpoint computations.

1.3. Mathematical Abstract Domains

1.3.1. Constant Propagation Domain

To perform precise static evaluations, this domain is designed to track exact, statically determinable constant values throughout the program's execution.

- **ConstantLattice:** This lattice defines a flat mathematical structure that maps concrete integers directly to themselves. It is bounded by a $\top$ element (representing an unknown or non-constant value) and a $\perp$ element (representing an unreachable state).
- **Constant Domain:** Handles direct abstract arithmetic. If two identical constants merge during control flow, the precision is retained ($c \sqcup c = c$). If two divergent constants merge, the domain safely degrades to uncertainty ($c_1 \sqcup c_2 = \top$). Arithmetic operations are evaluated exactly only if both operands are known constants.

1.3.2. Sign Domain

To perform basic numerical verification, it was implemented a sign analysis domain designed to safely approximate the values of variables by tracking only their mathematical sign.

- **SignLattice:** This lattice defines a flat mathematical structure mapped to five distinct elements:
 - **POS, NEG, ZERO:** Represent strictly positive (> 0), strictly negative (< 0), and exactly zero ($= 0$) values.

- **TOP:** An unknown state. In abstract arithmetic, unpredictable operations (e.g., $POS + NEG$) must evaluate to TOP to maintain a sound analysis.
- **BOTTOM:** Represents an unreachable program state or uninitialized data.
- **Sign Domain:** Handles the strict abstract arithmetic and propagation rules. It evaluates mathematical operations safely (e.g., $POS * NEG$ degrades to NEG) and uses boolean branching guards (e.g., $\text{if } x > 0$) to actively refine and restrict variable states during the execution flow.

1.3.3. Parity Domain

To abstract numerical variables based on their divisibility by two, this domain is implemented to verify memory alignments and specific algorithmic invariants.

- **ParityLattice:** A flat structure containing two core elements: *EVEN* and *ODD*. It is bounded by $\top$ (uncertain parity) and $\perp$ (unreachable state).
- **Parity Domain:** Evaluates mathematical operations using modular arithmetic logic. Addition alternates states symmetrically ($EVEN + ODD = ODD$), while multiplication by an even number acts as a mathematical annihilator ($EVEN \cdot \top = EVEN$), successfully recovering precision even when one operand is entirely unknown.

1.3.4. Interval Domain

To detect critical numerical vulnerabilities such as out-of-bounds memory accesses and integer overflows, this domain is engineered to track the absolute lower and upper boundaries of numerical variables.

- **IntervalLattice:** Represents abstract states as closed intervals $[a, b]$, where the bounds can extend to infinity $(-\infty, +\infty)$. The $\top$ element is the completely unbounded interval $[-\infty, +\infty]$, while $\perp$ is the mathematically empty set $(\emptyset)$.
- **Interval Domain:** Applies boundary arithmetic to approximate operations (e.g., $[a, b] + [c, d] = [a + c, b + d]$). Crucially, to guarantee the mathematical termination of the fixpoint computation inside unbounded loops, this domain implements a **Widening Operator** (∇) that artificially pushes expanding boundaries to $+\infty$, sacrificing peak precision to ensure computational safety.

1.3.5. Taint Three Level Domain

To perform security and data-integrity verification, we implemented a taint analysis domain designed to track the flow of untrusted data from source inputs to critical execution sinks (Information Flow Analysis).

- **TaintThreeLevelsLattice:** This lattice defines a security clearance hierarchy mapped to three distinct levels, ordered by their level of contamination:
 - **CLEAN:** Safe, sanitized, or internally generated data.
 - **TOP:** An unknown state. In a security context, unknown data must be treated as potentially dangerous to maintain a sound analysis.
 - **TAINTED:** Explicitly untrusted data originating from an external source (e.g., HTTP requests, database inputs).
- **TaintThreeLevels:** Handles the strict propagation rules. If a **CLEAN** variable interacts with a **TAINTED** variable in any binary expression (e.g., string concatenation), the resulting state immediately degrades to **TAINTED**. Data can only move back down the lattice to **CLEAN** if it

explicitly passes through a recognized sanitizer function. This domain directly powers our Information Leakage and SQL Injection semantic checkers.

2. Widening Operator (∇)

LiSA computes the abstract semantics of an IMP program by iteratively evaluating transfer functions across the Control Flow Graph until it converges on a stable solution, known as a **fixpoint**.

2.1. Ascending Chain Condition (ACC)

For finite domains or domains that naturally satisfy the **Ascending Chain Condition (ACC)**, such as our Taint or Extended Sign domains, the worklist algorithm is guaranteed to terminate because the abstract values cannot grow infinitely. The maximum height of the lattice is strictly bounded (e.g., $Bottom \rightarrow > 0 \rightarrow Top$). Consequently, applying the standard Least Upper Bound (LUB) operator is sufficient to guarantee fixpoint convergence in a finite number of iterations.

However, introducing the **Float Interval** domain breaks the ACC. Floating-point or rational chains can grow infinitely within a loop, such as a geometric sequence (e.g., $0.5 \sqsubseteq 0.75 \sqsubseteq 0.875 \dots$) that continuously approaches an asymptotic upper bound without ever reaching a stationary state. To force convergence at loop heads, we configured and evaluated three iterations of a **Widening Operator (∇)**:

- **Naive Widening:** Inherited from basic integer implementations, this operator instantly forces any unstable moving bound to jump straight to $\pm\infty$. While sound and terminating, it is hopelessly imprecise for real-world calculations.
- **Single-Threshold Widening (K):** This strategy maintains precise bounds as long as they stay within a predefined threshold $[-K, K]$, jumping to $\pm\infty$ only when they escape. While effective for integers, we discovered a major theoretical limitation for float values: an infinite geometric ascending chain can climb indefinitely *inside* the threshold $[-K, K]$ without ever escaping, violating the formal widening property that requires sequences to become ultimately stationary.
- **Multi-Threshold Widening (Final Implementation):** To fix the float termination flaw, we designed a widening operator backed by two sorted arrays of boundary thresholds (POS_THRESHOLDS and NEG_THRESHOLDS). When a bound moves in an unbounded direction, it immediately jumps to the next highest positive threshold (or lowest negative threshold) that is greater than or equal to the new value, or to $\pm\infty$ if it exceeds all levels. This guarantees termination across all real-valued chains by allowing a finite number of escalations, successfully achieving mathematical stationarity at the loop head.

2.2. Narrowing Operator (Δ)

The theoretical counterpart to widening is the **Narrowing Operator (Δ)**. After widening has pushed a bound to infinity, narrowing is used to refine the result by applying the loop guard constraints (e.g., the condition $x < 5.0$). If implemented via specific state refinement methods (such as `assumeBinaryExpression`), narrowing "pulls back" the infinite bound to a precise value like $[0.0, 5.0]$, reclaiming the precision lost during widening without sacrificing termination.

Our custom domains do not implement branch refinement via the `assumeBinaryExpression` method. Because of this, loop guards (such as `while (x < 1.0)`) do not dynamically filter or narrow down the abstract intervals of variables inside the loop body.

While the analysis remains completely **sound**, safely over-approximating all possible execution trajectories, the lack of branch refinement explains why our float loop tests eventually widen up to $+\infty$.

at the upper bound. Since narrowing operators are designed to act as a second descending phase to recover precision lost by widening at loop boundaries, omitting `assumeBinaryExpression` meant that a narrowing phase would have nothing to narrow against. This limitation is documented as a precision trade-off and serves as a primary candidate for future development.

3. Abstract Domains

Abstract interpretation relies on mapping concrete program semantics to abstract mathematical structures (Complete Lattices). In this section, we present the abstract domains we built and integrated into our static analyzer. We outline the mathematical structure, evaluation logic, and architectural engineering choices for each domain, progressing from foundational analyses up to our custom, complex implementations.

3.1. Extended Sign Domain (ExtendedSign & ExtendedSignLattice)

The standard Sign domain is inherently limited by its flat lattice structure. Because its concrete abstractions (POS, NEG, ZERO) are mutually exclusive and incomparable, any operation that yields ambiguity (such as adding a positive number to a negative one) forces the analyzer to immediately degrade to TOP, resulting in a massive loss of precision.

To mitigate this precision loss, we engineered an extended sign domain that tracks specific combinations of these states.

3.1.1. ExtendedSignLattice

Rather than defining a flat enumeration, we structured the ExtendedSignLattice as a boolean lattice isomorphic to the powerset of the base signs: $\mathcal{P}(\{< 0, = 0, > 0\})$. Every abstract state is internally represented by a triplet of boolean flags: includesNeg, includesZero, includesPos. This architectural choice makes operations extremely CPU-efficient.

This topological design automatically generates exactly $2^3 = 8$ distinct lattice elements:

1. **BOTTOM** (F, F, F): The unreachable state.
2. **LT_ZERO** (T, F, F): Strictly negative (< 0).
3. **EQ_ZERO** (F, T, F): Exactly zero ($= 0$).
4. **GT_ZERO** (F, F, T): Strictly positive (> 0).
5. **LE_ZERO** (T, T, F): Less than or equal to zero (≤ 0).
6. **NE_ZERO** (T, F, T): Not equal to zero ($\neq 0$).
7. **GE_ZERO** (F, T, T): Greater than or equal to zero (≥ 0).
8. **TOP** (T, T, T): Maximum uncertainty.

This structural choice resolves the lattice ordering operations. The Least Upper Bound (lubAux) is computed safely and instantaneously by performing a logical OR operation between the corresponding flags of the two operands. Similarly, the partial order relation $a \sqsubseteq b$ (lessOrEqualAux) is verified by ensuring that every active flag in a is also active in b .

3.1.2. ExtendedSign

The true power of this powerset representation emerges during the semantic evaluation of arithmetic operations within the ExtendedSign class. Instead of exhaustively defining transition matrices for every possible pair of states, abstract arithmetic is evaluated through highly efficient boolean logic.

Consider the abstract addition operator (add):

```
boolean resZero = (this.includesZero && other.includesZero)
    || (this.includesPos && other.includesNeg)
    || (this.includesNeg && other.includesPos);
```

This single logical statement captures the fundamental principle of abstract sound over-approximation. It mathematically means that the result of an addition can only be exactly zero if both operands are zero, or if a potentially positive value is added to a potentially negative one (which introduces ambiguity, thus requiring the zero flag to be set to preserve soundness).

Multiplication (mul) and Division (div) follow similar boolean derivations. Specifically for division, the domain implements strict safety checks: if the divisor lattice does not contain any non-zero elements (`!other.includesPos && !other.includesNeg`), the operation correctly collapses to **BOTTOM**, effectively catching implicit division-by-zero anomalies before they propagate further into the control flow graph.

3.2. Float Interval Domain (FloatInterval & FloatIntervalLattice)

The Extended Sign domain is great for tracking if a variable is positive or negative, but it does not track its actual size (magnitude). To perform a more precise numerical analysis, we implemented the **Float Interval Domain**. This domain tracks the continuous bounds of a variable, representing it as an interval $[l, u]$.

3.2.1. FloatInterval

The `FloatInterval` class implements the `BaseNonRelationalValueDomain` interface. It defines how math operations work on continuous intervals to maintain **soundness**:

- **Addition and Subtraction:** Addition is simple ($[l_1 + l_2, u_1 + u_2]$), but subtraction requires cross-computation to be safe. The result must be $[l_1 - u_2, u_1 - l_2]$ to cover the worst-case scenario.
- **Multiplication and Division:** Because the bounds can be positive or negative, we cannot just multiply the extremes. Our code calculates all four possible boundary combinations ($l_1 l_2, l_1 u_2, u_1 l_2, u_1 u_2$) and extracts the absolute minimum and maximum to form the new interval.
- **Safety Checks (Division by Zero):** In the `DivisionOperator`, we added strict rules. If the divisor is exactly zero ($[0, 0]$), the division returns **BOTTOM** ($\perp$), meaning the program will crash and the next instructions are unreachable. If the divisor spans across zero (e.g., $l \leq 0$ and $u \geq 0$), the operation returns **TOP** ($\top$). This is an over-approximation, but it is necessary to prevent undefined behaviors and keep the analysis sound.

During initial prototyping, we observed that parsing source code literals like 0.1 using standard floating-point conversions created IEEE 754 rounding artifacts (e.g., evaluating to 0.1000000014...). We resolved this by routing all constant evaluations through Java's `BigDecimal` class to capture the exact string form, yielding clean, mathematically sound intervals like $[0.1, +\text{Inf}]$.

3.2.2. FloatIntervalLattice

In our `FloatIntervalLattice` class, we represent the intervals using `MathNumber`, which allows us to manage floating-point values and extreme limits like $+\infty$ and $-\infty$.

Working with real numbers creates a big problem for the analysis. Floating-point numbers are infinitely dense, so this domain violates the Ascending Chain Condition (ACC). For example, a simple loop

could generate an infinite chain of intervals: $[0,0] \sqsubseteq [0,0.1] \sqsubseteq [0,0.2] \dots$ and the analysis would never reach a fixpoint.

To solve this and guarantee termination, we implemented a **Widening Operator** (∇). However, jumping directly to $+\infty$ would destroy too much precision. To keep the analysis useful, we engineered a **Multi-Threshold Widening**. As you can see in our code, we defined specific arrays of thresholds (e.g., 1, 10, 100, 1000).

```
// Thresholds for widening
private static final MathNumber[] POS_THRESHOLDS = {
    new MathNumber(1),
    new MathNumber(10),
    new MathNumber(100),
    new MathNumber(1000)
};
private static final MathNumber[] NEG_THRESHOLDS = {
    new MathNumber(-1000),
    new MathNumber(-100),
    new MathNumber(-10),
    new MathNumber(-1)
};
```

If a variable's interval boundary continuously grows during a loop iteration, our widening operator detects the instability and forces the bound to jump to the next available scalar threshold from the array. If it surpasses the maximum defined threshold, it escalates directly to $\pm\infty$.

It is important to note that, because we did not implement a **Narrowing Operator** (Δ), the analyzer cannot filter and refine the interval when the loop finishes. This means that if a boundary reaches infinity during the widening phase, it will permanently stay at infinity. Because the number of possible steps is limited by the array, we guarantee fixpoint convergence and termination, but with an unavoidable loss of precision at the end of large loops.

3.3. Sign-Taint Combined Domain (SignTaint & SignTaintLattice)

In modern software, knowing the mathematical value of a variable is often not enough. We also need to know if that data is safe to use. To analyze both the numerical properties and the security level at the same time, we implemented the **SignTaint Domain**. This is a composite abstract domain that combines our Extended Sign with a Three-Level Taint domain (CLEAN, SUSPICIOUS, TAINTED) using a **Cartesian Product**.

3.3.1. SignTaintLattice

Our SignTaintLattice is built as a Cartesian Product. This means that every abstract state is represented as a pair of values, one for the sign and one for the taint level (ExtendedSignLattice, TaintThreeLevelsLattice). For example, a state can be (≥ 0 , TAINTED).

In the lattice implementation, operations like the Least Upper Bound (lubAux) and the partial order (lessOrEqualAux) are calculated **component-wise**. This means the analyzer compares the signs together and the taints together, completely independently.

However, we implemented a strict normalization rule for the **BOTTOM** ($\perp$) state. In lattice theory, this specific optimization is known as a **Smash Product**. As you can see in the constructor of our code, if either the sign or the taint evaluates to BOTTOM, we force the entire pair to collapse into a canonical (BOTTOM, BOTTOM) state. This makes sense because if a variable is mathematically unreachable (e.g., after a division by zero), its security level does not matter anymore because the program will

crash. Applying this Smash Product logic optimizes the domain and streamlines the Java equals checks. For all the other valid states, we maintained a plain Cartesian product.

```
SignTaintLattice(ExtendedSignLattice sign, TaintThreeLevelsLattice taint) {
    if (sign.isBottom() || taint.isBottom()) {
        this.sign = ExtendedSignLattice.BOTTOM;
        this.taint = TaintThreeLevelsLattice.BOTTOM;
    } else {
        this.sign = sign;
        this.taint = taint;
    }
}
```

Instead of statically pre-allocating all possible coordinate combinations (e.g., the 29 distinct valid states) at boot time, we optimize memory by only defining static TOP and BOTTOM instances. Intermediate Cartesian states are generated dynamically during the analysis. Because of this, our domain relies on the overridden Java equals and hashCode methods to correctly perform structural comparisons between different abstract states.

3.3.2. SignTaint

The evaluation methods inside this class act as stateless dispatchers. They seamlessly delegate the workload to the underlying domains. For example, evalBinaryExpression independently invokes the binary evaluation of the sign domain and the taint domain, fusing their results back into a new Cartesian pair.

While this approach is easy to implement and mathematically **sound**, it has a well-known limitation. Because it is a plain Cartesian Product, there is no **reduction operator (ρ)** to share information between the two domains. For example, if we multiply a TAINTED variable by 0, the sign domain correctly evaluates to =0, but the taint domain blindly keeps the TAINTED state. The two domains do not "talk" to each other to understand that the data was mathematically destroyed. This generates some **False Positives** during the analysis (as we explained in the "Zero-Taint" test case), but it is a necessary compromise to keep the analyzer safe and sound.

4. Semantic Checkers

In the LiSA framework, semantic checkers operate as visitors traversing the Control Flow Graph (CFG) after the fixpoint computation has stabilized. By interrogating the computed abstract state at specific statement nodes, these checkers detect runtime errors and security vulnerabilities.

As our primary custom property checker, we engineered the **ArrayBoundsPentagonChecker**. This checker is designed to prevent critical memory corruption vulnerabilities, specifically **Buffer Underflows** and **Buffer Overflows**, by leveraging the relational capabilities of the Pentagon abstract domain.

4.1. Architectural Integration and State Extraction

The checker integrates into the analysis lifecycle by operating as a post-analysis visitor. Through the visit method, it traverses the Control Flow Graph (CFG) and specifically intercepts nodes of type `IMPArrayAccess`. These nodes represent array read or write operations in the source code, such as `arr[i]`.

Before performing any mathematical validation, the checker must translate the raw program syntax into concepts that the abstract domains can understand. To do this, it leverages LiSA's AnalyzedCFG. Because the fixpoint computation is already complete at this stage, the CFG contains the fully calculated results for every instruction. The checker queries this graph to extract the exact `AnalysisState` immediately following the evaluation of the array and the index.

Inside LiSA, the `AnalysisState` is not just one single block of data. It is divided into two separate parts that usually work independently:

- **The Value Environment (Pentagon Domain):** This part only handles numbers and math relations (like $i \geq 0$).
- **The Heap Environment:** This part only handles the computer memory, keeping track of pointers, objects, and their fields.

To check if an array access is safe, our checker must force these two separate worlds to talk to each other. It must extract the array length from the Heap memory, and then use the Pentagon domain to verify the mathematical math condition.

LiSA abstracts language-specific syntax into a universal intermediate representation called **Symbolic Expressions**. To build our security checks, the checker extracts two specific symbolic entities from the state:

- `indexSymbolic`: The mathematical representation of the index variable (which allows the domain to read its numeric interval, e.g., the value of `i`).
- `arraySymbolic`: The memory pointer representing the array object (which allows the domain to traverse the Heap to find its length).

By successfully extracting and isolating these two symbolic elements, the checker builds the necessary foundation to execute our strict underflow and overflow relational queries.

4.2. Lower Bound Validation (Underflow Prevention)

To verify that an array index is never negative ($i \geq 0$), the checker constructs a syntactic query and evaluates it against the abstract domain.

1. **Syntactic Construction:** The checker instantiates a Constant zero. In LiSA's architecture, a Constant represents the syntactic text of a number, differentiating it from purely mathematical domain objects (like MathNumber).
2. **Predicate Generation:** A BinaryExpression is constructed to form the logical question. This expression strictly requires five parameters to define the abstract syntax tree node: the return type (BoolType), the left operand (indexSymbolic), the right operand (the Constant zero), the operator (ComparisonGe for $\geq$), and the physical source location for precise error reporting.
3. **Semantic Interrogation:** The checker acts as a bridge, passing this syntactic BinaryExpression to the abstract domain's satisfies method.

The domain mathematically evaluates the predicate and returns a Satisfiability result. If the evaluation yields NOT_SATISFIED, the index is definitively negative, and a strict underflow warning is emitted. If the result is UNKNOWN (e.g., the index interval spans across zero), a possible underflow warning is issued.

4.3. Upper Bound Validation and Heap Traversal

Validating the upper bound ($i < arr.length$) introduces significant architectural complexity because an array's length is not a static constant; it is dynamically stored within the heap memory.

To resolve the array length, the checker simulates memory traversal using a three-step symbolic extraction process:

1. **HeapDereference:** The analyzer must resolve the memory pointer. We instantiate a HeapDereference object using the arraySymbolic pointer. This operation mathematically models the action of following a memory address from the local Stack into the dynamically allocated Heap to access the core object instance.
2. **Targeting the Property:** Once the object in the Heap is resolved, we must isolate the specific field that holds the array's capacity. We instantiate a symbolic Variable explicitly named "len". In the IMP language semantics, this variable conceptually represents the internal memory slot allocated for the array's size.
3. **AccessChild:** These two elements are combined into an AccessChild symbolic expression. This creates a complete structural representation that tells the analyzer exactly how to traverse the memory tree: *dereference the pointer, then read the child field named "len"*. This mathematically represents the retrieval of arr.length.

With the length dynamically extracted, the checker constructs a new BinaryExpression utilizing the ComparisonLt (<) operator, asking the fundamental question: *Is the index strictly less than the length?* This predicate is then passed to the Pentagon domain's satisfies method to mathematically prove the safety of the access.

4.4. The Relational Advantage of the Pentagon Domain

The final satisfies query for the upper bound demonstrates the critical superiority of relational domains over non-relational domains.

While we built the Extended Sign and Float Interval domains for this project, we decided to use LiSA's native PentagonLattice as the mathematical foundation for our array bounds verification.

To understand why we chose it, we have to look at the limits of standard domains. A standard Interval domain is **non-relational**. It tracks the value of each variable completely independently. For example, it might know that $i \in [0,10]$ and $len \in [10,10]$, but it does not track any direct relationship between the two. This is very fast for the CPU, but it loses a lot of precision during if or while conditions involving two variables.

The Pentagon domain, instead, is a **weakly relational domain**. This means it maintains two pieces of abstract information at the same time:

1. The standard non-relational interval for every single variable.
2. A relational graph that tracks **strict inequalities** between pairs of program variables, in the mathematical form $x < y$.

This specific structure makes it the perfect tool for checking array limits. When the analyzer evaluates a loop guard like while ($i < arr.len$), the Pentagon domain captures this condition and permanently stores the relation $i < len$ inside its abstract memory. As we will see in our checker's results, even if the non-relational interval of i explodes to $+\infty$ due to the widening operator, the relational constraint $i < len$ remains mathematically proven, keeping the analysis precise and safe.

4.5. Experimental Validation and Results

4.5.1. Test Methodology

To empirically validate the implementation of the *Pentagon* abstract domain and the *ArrayBoundsPentagonChecker*, a comprehensive test suite was designed using the IMP language. The custom test program (array.imp) was structured to simulate various real-world array access scenarios, effectively stressing both the non-relational (intervals) and strictly relational components of the domain.

The test cases were divided into three main categories:

- **Statically Safe Operations:** Baseline accesses with hardcoded, valid indices to ensure no false alarms are triggered under normal conditions.
- **Definite Violations:** Methods such as definiteUnderflow and definiteOutOfBounds, where indices strictly fall outside the mathematical bounds of the array ($i \notin [0, length - 1]$).
- **Potential Violations:** Parameter-dependent methods where inputs are only partially sanitized (e.g., verifying the lower bound but ignoring the upper bound), forcing the analyzer to handle uncertainty.
- **Relational Safety (The Pentagon Test):** A specific method, pentagonSafeLoop, designed to iterate over an array using a while ($i < a.len$) condition. This serves as the ultimate benchmark for the relational domain.

```

class ArrayBoundsTest {

    // 1. SECURE static Case
    safeAccess() {
        def a = new int[10];
        def i = 5;
        a[i] = 42;
        return a;
    }

    // 2. DEFINED Error: Underflow
    definiteUnderflow() {
        def a = new int[10];
        def i = -2;
        a[i] = 42;
        return a;
    }

    // 3. POSSIBLE Error: Underflow
    possibleUnderflow(param) {
        def a = new int[10];
        if (param < 5) {
            a[param] = 42;
        }
        return a;
    }

    // 4. DEFINED Error: Out-of-bounds
    definiteOutOfBounds() {
        def a = new int[10];
        def i = 10;
        a[i] = 42;
        return a;
    }

    // 5. POSSIBLE Error: Out-of-bounds
    possibleOutOfBounds(param) {
        def a = new int[10];
        if (param >= 0) {
            a[param] = 42;
        }
        return a;
    }

    // 6. Test for Pentagon
    pentagonSafeLoop() {
        def a = new int[10];
        def i = 0;

        while (i < a.len) {
            a[i] = 42;
            i = i + 1;
        }
        return a;
    }

    main() {
        this.safeAccess();
        this.definiteUnderflow();
        this.possibleUnderflow(3);
        this.definiteOutOfBounds();
        this.possibleOutOfBounds(5);
    }
}

```

```

    this.pentagonSafeLoop();
}
}

```

4.5.2. Results Discussion

The static analysis executed successfully, processing the 7 Control Flow Graphs (CFGs). The analyzer behaved exactly as expected, raising four specific warnings corresponding only to the vulnerable methods.

Identification of Definite Violations

The analyzer accurately captured the two cases where out-of-bound errors were mathematically guaranteed (definiteUnderflow with a fixed index of -2, and definiteOutOfBounds with an index of 10). In these scenarios, the abstract domain evaluates the variables as exact intervals (e.g., $[-2, -2]$). When the checker compares this interval against the safe boundaries, the mathematical thresholds are strictly exceeded, and the domain correctly evaluates the satisfiability check as NOT_SATISFIED.

Handling of Uncertainty (Unknown Parameters)

In the possibleUnderflow method, the external input is only partially constrained by the `if (param < 5)` guard. Because of this, the abstract domain evaluates the variable's state as the interval $[-\infty, 4]$. When our checker asks if this interval is strictly ≥ 0 , the domain successfully recognizes the mathematical ambiguity: values like 3 are safe, but values like -1 are not. It correctly evaluates the condition as UNKNOWN and appropriately emits the "Possible underflow" warning.

Success of the Relational Domain (Zero False Positives)

This represents the most important result of our experiment. In the `pentagonSafeLoop` method, the array access occurs within a while loop. If we had used a standard non-relational domain (such as classic Intervals), the widening operator would inevitably push the index's upper bound to $+\infty$ to prevent an infinite loop. This would trigger a massive false alarm when checking the array length. The Pentagon domain, however, successfully retains the strict relational invariant ($i < a.length$) directly inside its abstract memory. This allows the analyzer to prove the array access safety at every iteration, completing the analysis without emitting any spurious warnings.

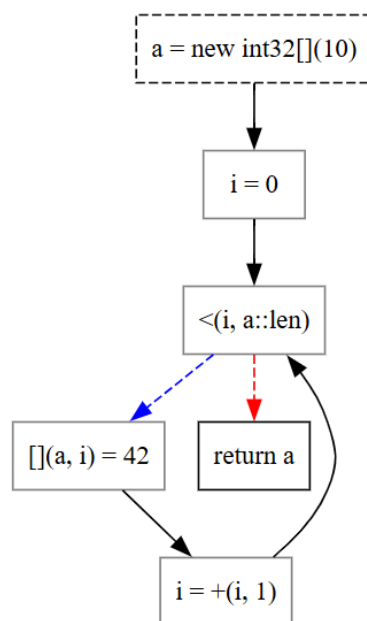


Figure 1: CFG `pentagonSafeLoop`

By inspecting the final abstract state at the return a node in the CFG (Figure 1), we observed an interesting result demonstrating LiSA's internal workings. The state dump reveals the following values:

- `i`: `[10, +Inf]`
- `heap[w]:heap`: `[10, 42]`

The value of the index `i` behaves perfectly according to the principles of abstract interpretation with state refinement (`assume`). At the loop header, the widening operator pushes the interval of `i` to `[0, +∞]`. Inside the loop body, the Pentagon domain uses the `assume(i < a.len)` expression to temporarily refine the state to `[0, 9]`, ensuring the array access is perfectly safe.

However, at the loop exit node (`return a`), the analysis follows the false branch, applying the negated guard `assume(i ≥ a.len)`. Intersecting the widened state `[0, +Inf]` with the condition `i ≥ 10` yields exactly `[10, +Inf]`. The `assume` successfully filtered out the values 0 through 9, leaving the upper bound mathematically unbounded.

Furthermore, the array abstraction evaluates to `heap[w]:heap`: `[10, 42]`. This provides empirical evidence of LiSA's **Monolithic Heap** representation (field-insensitive). Rather than tracking the array's length field and its elements as separated variables, the monolithic heap abstracts the entire object into a single summary variable. The framework applies the Least Upper Bound (LUB) between the array's initial capacity (10) and the assigned elements (42), mathematically merging them into the interval `[10, 42]`.

Conservative Behaviour of the Framework

An interesting behavior was observed during the analysis of the `possibleOutOfBounds` method. In the IMP test program, this method is logically designed to represent a potential violation, because the input parameter is only constrained from below (`param ≥ 0`), meaning its abstract interval is `[0, +Inf]`.

When our checker evaluates the upper bound predicate (`indexSymbolic < lengthExpr`), it compares this unbounded interval against the fixed array length of 10. Mathematically, this condition is ambiguous because some concrete values (like 5) are safe, while others (like 15) cause an overflow. However, the underlying LiSA framework adopts a strict **Conservative Behaviour** (safe-by-default policy). When encountering an upper bound pushed to `+∞`, the framework automatically evaluates the satisfiability check as `NOT_SATISFIED` instead of `UNKNOWN`.

Because the abstract domain returns a definitive failure token, our semantic checker reacts accordingly and triggers the stricter warning: *"Array index definitely exceeds length. Out-of-bounds"*. This behavior proves that the framework aggressively prioritizes security and buffer overflow prevention, treating any completely unconstrained upper limit as a certain runtime violation to guarantee absolute soundness.

Next, we can view the report that has been created:

```
"warnings" : [ {
  "message" : "[inputs/peer-programs/taintthreelevels/array.imp':15:10] on
'untyped ArrayBoundsTest::definiteUnderflow(ArrayBoundsTest* this)':
[EXPRESSION] Array index is definitely negative. Underflow"
}, {
  "message" : "[inputs/peer-programs/taintthreelevels/array.imp':23:14] on
'untyped ArrayBoundsTest::possibleUnderflow(ArrayBoundsTest* this, unt
yped param)': [EXPRESSION] Array index may be negative. Possible underflow"
}, {
  "message" : "[inputs/peer-programs/taintthreelevels/array.imp':32:10] on
'untyped ArrayBoundsTest::definiteOutOfBounds(ArrayBoundsTest* this)':
[EXPRESSION] Array index definitely exceeds length. Out-of-bounds"
```

```
}, {  
  "message" : "[ 'inputs/peer-programs/taintthreelevels/array.imp':40:14] on  
'untyped ArrayBoundsTest::possibleOutOfBounds(ArrayBoundsTest* this, untyped  
param)': [EXPRESSION] Array index definitely exceeds length. Out-of-bounds"  
} ]
```

5. Results on the Custom Tests

Each program below is framed as a small but recognizable real-world pattern and is designed to drive every verdict the corresponding checker can emit.

5.1. Division by zero

The four programs model arithmetic where a denominator may become zero.

- average(sum, count):** a mean over a caller-supplied count. The divisor is an unbounded parameter, so the analyzer emits a single "may be possible division by zero" warning, as expected.


```
class statistics {
    average(sum, count) {
        def avg = sum / count;
        return avg;
    }
}
```
- rescale:** computes $\text{range} = \text{max} - \text{min}$ with $\text{min} == \text{max} == 10$, then divides by range. The analyzer proves the divisor is exactly zero and raises a "definite division by zero", as expected.


```
class normalizer {
    rescale(value) {
        def min = 10;
        def max = 10;
        def range = max - min;
        def scaled = value / range;
        return scaled;
    }
}
```
- accumulate:** divides by a counter that only ever grows inside a loop. The analyzer proves the divisor is strictly positive and emits no warning, as expected.


```
class accumulator {
    accumulate() {
        def i = 1;
        def sum = 0;
        while (i < 100) {
            sum = sum + i;
            i = i + 1;
        }
        def avg = sum / i;
        return avg;
    }
}
```
- zeroFactor** (unexpected result): computes $\text{total} = \text{score} \times 0$ and then $100 / \text{total}$. We designed this as a definite division by zero, since multiplying by zero is mathematically zero. The analyzer instead reports "may be possible", because score is an unbounded

parameter $[-\infty, +\infty]$ and the interval product $[-\infty, +\infty] \times [0, 0]$ is the indeterminate form $\infty \cdot 0$, which the domain evaluates to TOP rather than $[0, 0]$.

The analyzer therefore cannot prove total is exactly zero, it only knows that zero is possible, so multiplying an unbounded value by zero loses the "exactly zero" fact in interval arithmetic.

```
class scoreCalculator {
    compute(score) {
        def weight = 0;
        def total = score * weight;
        def avg = 100 / total;
        return avg;
    }
}
```

5.2. Numerical Overflow and Underflow

A single program bundles three classes and is analyzed at 8-, 16-, and 32-bit bounds.

- **geometry.area(width, height):** multiplies two unbounded parameters. The result interval can exit the representable range, so the analyzer correctly raises an over/underflow warning at every bit width.

```
class geometry {
    area(width, height) {
        def a = width * height;
        return a;
    }
}
```

- **smallCalc.sumHours():** performs small, bounded constant arithmetic. The result stays within range and produces no warning, as expected.

```
class smallCalc {
    sumHours() {
        def fixed = 5;
        def base = fixed + fixed;
        def total = base * 3;
        return total;
    }
}
```

- **processor.sumScores()** (unexpected result): accumulates +50 per iteration inside a loop bounded by $i < 100$. We expected this to overflow at 8-bit but be safe at 32-bit, since the true value is $50 \times 100 = 5000$, far inside the int range. In reality it warns at every bit width, because widening pushes the accumulator to $[0, +\infty]$: the loop guard $i < 100$ is never used to bound total (no branch refinement), so the precise 5000 is lost. This is the widening precision cost producing a borderline false positive, and it is the motivation for pairing narrowing with `assumeBinaryExpression`.

```
class processor {
    sumScores() {
        def total = 0;
        def i = 0;
        while (i < 100) {
```

```

        total = total + 50;
        i = i + 1;
    }
    return total;
}
}

```

5.3. String Prefix and Suffix

One program drives both the HTTP-prefix and the dot-com-suffix checkers. These checkers inspect the string arguments of every call and flag a call whose argument starts with `http://https://` (prefix) or ends with `.com` (suffix).

- **webDownloader** is called with `"http://example.com"`: the prefix checker flags it (HTTP URL) and the suffix checker also flags it (it ends in `.com`).

```

class webDownloader {
    download(url) {
        return url;
    }

    fetch() {
        this.download("http://example.com");
    }
}

```

- **emailService** is called with `"user@example.com"`: the prefix checker stays silent, while the suffix checker flags the `.com` ending.

```

class emailService {
    sendEmail(addr) {
        return addr;
    }

    notify() {
        this.sendEmail("user@example.com");
    }
}

```

- **fileLoader** is called with `"config.txt"`: both checkers stay silent.

```

class fileLoader {
    readFile(path) {
        return path;
    }

    load() {
        this.readFile("config.txt");
    }
}

```

The measured result is one prefix warning and two suffix warnings, exactly the intended split. The suffix checker correctly fires on both the URL and the email address (both terminate in .com) and remains silent on the file path.

5.4. Three-Level Taint

The three programs reproduce three classic vulnerability classes and exercise all three taint verdicts.

- **userLogin.handleLogin (SQL injection)**: a tainted source flows into a database sink with no sanitizer. The analyzer raises a Definite Warning.

```
class userLogin {
    getRequestParam() {
        return 0;
    }

    runQueryDB(v) {
    }

    handleLogin() {
        def username = this.getRequestParam();
        this.runQueryDB(username);
    }
}
```

- **commentForm.submitComment (stored XSS)**: the source passes through an escapeHtml sanitizer before reaching the rendering sink. The taint is cleared and the analyzer emits no warning.

```
class commentForm {
    readRawInput() {
        return 0;
    }

    escapeHtml(s) {
        return 0;
    }

    renderPage(v) {
    }

    submitComment() {
        def raw = this.readRawInput();
        def safe = this.escapeHtml(raw);
        this.renderPage(safe);
    }
}
```

- **shellExec.run(trusted)** (command injection): the source is sanitized only inside a conditional branch (if (trusted)). Because the sanitizer is not guaranteed on all paths, the analyzer raises a Possible Warning.

```
class shellExec {
    readUserCommand() {
```

```

        return 0;
    }

    sanitize(cmd) {
        return 0;
    }

    executeCommand(c) {
    }

    run(trusted) {
        def cmd = this.readUserCommand();
        if (trusted) {
            cmd = this.sanitize(cmd);
        }
        this.executeCommand(cmd);
    }
}

```

Two setup choices underpin these programs. First, the source / sanitizer / sink roles are attached programmatically: each test annotates the relevant CFGs through `cfg.getDescriptor().addAnnotation(...)` rather than with inline source-level annotations.

Second, the source, sanitizer, and sink methods have empty (placeholder) bodies (`return 0;` or `{}`). This is intentional and sound: taint propagation is driven entirely by the annotations, not by what the functions compute: a source forces its result to TAINTED regardless of its body, a sanitizer forces CLEAN, and sinks are not introspected.

6. Flow Graphs and Abstract State Analysis

In this chapter, we use Control Flow Graphs to visually explore how LiSA reaches a fixed point. By extracting the Control Flow Graphs (CFGs) generated during the analysis, we can observe the step-by-step propagation of abstract states across program statements.

Rather than presenting the entire graph for every test case, we have selected specific, highly representative nodes from our IMP test programs. These case studies visually demonstrate how our custom domains handle loop convergence, data sanitization, and relational constraints.

6.1. Numerical Analysis: Loop Convergence and Widening

To demonstrate the numerical behavior and termination properties of our FloatInterval domain, we analyze a continuous increment loop bounded by a conditional guard (while ($x < 1.0$)).

```
test1() {
  def x = 0.0;
  while (x < 1.0) {
    x = x + 0.1;
  }
  def y = x;
  if (y < 1) {
    x = -0.5;
  }
  return x;
}
```

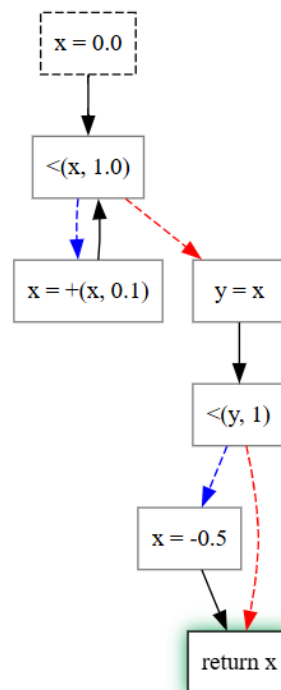


Figure 2: CGF test1 Float Interval Domain

- **Observation:** The analysis of this block highlights the interaction between the Widening operator (∇) at the loop head and the absence of branch refinement (assumeBinaryExpression). As observed in the abstract state output, the variables undergo an

intentional over-approximation to mathematically guarantee the termination of the fixpoint computation over real-valued chains.

- **State Analysis:** Before the loop, the variable x is initialized exactly at $[0.0, 0.0]$. However, observing the final post-state at the return x node, the environment maps y to $[0.0, +\text{Inf}]$. Because the domain currently lacks a narrowing operator and does not filter the state through the $y < 1$ condition, the multi-threshold widening safely escalates the upper bound to $+\infty$ to prevent an infinite analysis loop. Consequently, the final state of x safely merges the initial widening with the conditional reassignment (-0.5), yielding the sound, over-approximated interval $[-0.5, +\text{Inf}]$. This proves the analyzer prioritizes mathematical stationarity over absolute precision in floating-point sequences.

6.2. Combined Domain Analysis: Joint Extended Sign and Taint Propagation

To demonstrate the joint behaviour of our SignTaint Cartesian product domain, we analyze a program in which an annotated tainted source feeds a loop that mixes the source value with a locally-tracked accumulator. The aim is to observe both halves of the lattice (the ExtendedSign sign component and the TaintThreeLevels taint component) moving through structurally non-trivial control flow (a while containing a nested if/else).

```
process(steps) {
  def raw = this.getUserInput();
  def acc = 1;
  def i = 0;
  while (i < steps) {
    if (raw > 0) {
      acc = acc + raw;
    } else {
      acc = 0 - acc;
    }
    i = i + 1;
  }
  return acc;
}
```

The method `getUserInput()` is marked with the `lisa.taint.Tainted` annotation in the test class, simulating an external input source.

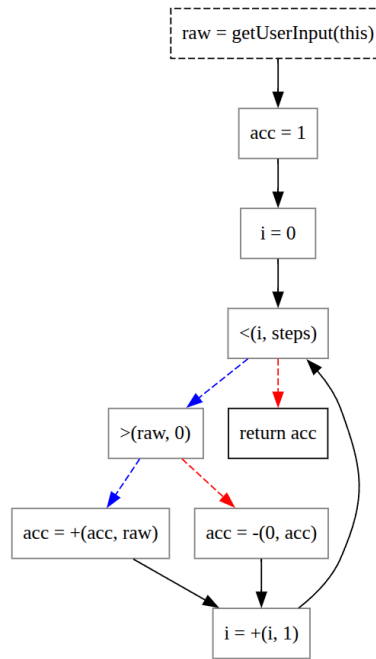


Figure 3: CFG process(steps) SignTaint domain

- Observation:** This graph highlights two distinct precision phenomena unfolding in parallel. The sign component undergoes the standard precision loss induced by branch joins inside the loop, while the taint component shows the irreversibility of taint flow under the three-level taint lattice.
- State Analysis:** Immediately after the annotated source call, the variable `raw` is abstracted to (TOP, Taint). The sign half is TOP because the annotation instructs the analyzer to treat the body as opaque. The taint half is Taint as imposed by the source annotation. Inside the loop's if-branch the assignment '`acc = acc + raw`' propagates taint into `acc` through arithmetic: the binary evaluation rule combines a CLEAN operand with a Taint operand to yield Taint. Once introduced, the taint cannot be eliminated within the loop, since no sanitizer is reached on either path. Symmetrically, the sign of `acc` widens from its initial GT_ZERO to TOP as the LUB across iterations covers both the '`accumulator + raw`' and '`0 - accumulator`' cases. The final post-state at `return acc` is (TOP, Taint), simultaneously witnessing the sign-side precision loss at the loop join and the persistence of taint across the entire control-flow graph.

7. Cross-Analysis of Peer Programs

We run checkers against IMP programs authored by other student groups. This phase allowed us to test our abstract domains against different coding styles and logical patterns not anticipated during our internal development.

For each semantic checker, we randomly selected ten peer-authored IMP files. We used the standard project configuration with `ContextBasedAnalysis` so that calls would carry caller information through the analysis. All the runs are in one JUnit class, `PeerAnalysisTest`, with a method per checker that loops through its list of files and dumps the result to `outputs/peer-analysis/`.

We tested the following checkers:

- `DivByZeroIntervalChecker`
- `OverflowIntervalChecker` (32 bit)
- `HTTPStringChecker`
- `DotComStringChecker`
- `TaintThreeLevelsChecker`

The Taint analysis required additional setup. Because different students used arbitrary nomenclature for their sources, sanitizers, and sinks, we manually inspected a subset of files to compile a dictionary of common identifiers (e.g., `getUserInput`, `runQueryDB`, `executeCommand`). We then developed a programmatic routine to dynamically identify and annotate the relevant CFGs with the appropriate Security Annotations before executing the fixpoint computation.

7.1. Div-by-zero: 880119-890558-divbyzero.imp

This file features eight methods (f0 to f7), each isolating a specific division-by-zero vulnerability pattern. Our checker successfully identified seven out of eight cases perfectly aligned with the program's intended semantics.

An interesting edge case occurred in method f5, which executes $1000 / b$ where b evaluates to 0, 10, or -5 depending on input parameters. The checker reported a *Definite Warning* for the zero branch. Because we used a context-sensitive analysis, the analyzer created a specific execution context where $b = 0$. From that precise perspective, the warning is technically guaranteed. However, looking at the method as a whole, emitting a *Possible Warning* might provide a more accurate representation of the global control flow.

7.2. Overflow: 894004_overflow.imp

Running the Interval domain against this file produced eleven warnings. One warning successfully identified a critical bug: the `accumulateHistogram()` method attempts to add two constants of 2,000,000,000, which explicitly overflows the 32-bit integer boundary. The remaining ten warnings were generated by accumulator updates inside loops (e.g., $i = i + 1$).

Because the standard Interval domain relies on the Widening operator to terminate the loop, the upper bound is invariably pushed to $+\infty$, triggering the checker at every iteration. While all warnings are mathematically sound (an infinite bound *could* overflow), this highlights a limitation in vulnerability reporting: without different severity levels or narrowing operators, valid warnings are often diluted by widening-induced noise.

7.3. HTTP Prefix Domain (872966_Strings.imp & 891184_strings.imp)

In the first file, our HTTPStringChecker raised seven accurate warnings on explicit variable flows, such as assigning a literal URL to a variable and passing it to a sink (e.g., `this.fetchResource(url)`).

Seven warnings were all raised on calls like:

```
def url = "http://dashboard.cluster.internal/v2/metrics";
this.fetchResource(url);    // line 44: warning
```

Direct literal → variable → sink. The Prefix domain proved extremely reliable for sequential string assignments.

An interesting observation for the HTTP prefix checker is in the following file:

- 891184_strings.imp
Zero warnings even though it builds URLs: the literal URLs are only stored in object fields (`this.apiUrl = "..."`) that are never passed to a call, while the strings that do reach a call (`fetchData`) are assembled by dynamic concatenation in `buildUrl()` (`url = url + "h" / "ws" + ...`), so the Prefix domain can't prove they start with `http://`.

7.4. Dot-com suffix: 880119-890558-dotcom-strings.imp

Eight warnings, mirror image of the prefix case. The interesting one is line 113, where the URL comes from an if-else:

```
if (region == 1) { cdn = "eu-west.cloudflare.com"; }
else if (region == 2) { cdn = "us-east.cloudflare.com"; }
else { cdn = "ap-south.cloudflare.com"; }
this.connectToServer(cdn);    // line 113: warning
```

All three branches end in `.com`, so the LUB of the three suffixes merged the states, successfully retaining the `.com` suffix information and accurately firing the warning. Had one branch ended in `.org`, the LUB would have lost the common suffix, allowing the checker to silently pass.

7.5. Three-level taint: 876957-taint.imp

The file has three classes (`SqlInjection`, `XssVulnerability`, `EmailService`) and each has three methods covering direct injection, mixed with partial sanitization, and conditionally sanitized. The checker produced six warnings: three definite (on the direct injections) and three possible (on the partial-sanitization cases), which matches exactly what the program is trying to test. When the source/sink names are correctly set up the taint analysis is reliable.

8. Key Findings and Technical Insights

8.1. Relational Domains are Mandatory for Array Loops

Our `ArrayBoundsPentagonChecker` proved that standard, non-relational domains (like Intervals) are virtually useless for checking array accesses inside loops. Because non-relational domains track every variable independently, they suffer from "amnesia." When analyzing `while (i < a.len)`, the Interval domain sees the index `i` growing and pushes it to $+\infty$ to avoid an infinite loop. It completely forgets the connection between `i` and `a.len`, causing a flood of false-positive Out-of-Bounds warnings. The **Pentagon domain**, being weakly relational, solves this entirely. It explicitly remembers the rule $i < a.len$ inside its memory. Even when the numeric value of `i` goes to infinity, the relational rule stays active, proving the access is safe and eliminating false alarms.

8.2. Floating-Point Termination Requires Sacrificing Precision

While building the `FloatInterval` domain, we encountered a major mathematical problem: floating-point numbers are infinitely dense. A variable could grow by tiny fractions (e.g., 0.1, 0.11, 0.111 ...) forever, meaning the analysis would never terminate (violating the Ascending Chain Condition).

We learned that to guarantee termination, we had to implement a **Multi-Threshold Widening** approach. This forces the analyzer to jump to predefined steps (e.g., 10, 100, 1000). However, if a variable grows beyond our highest threshold, the widening operator will eventually push the value to $+\infty$.

We accepted this extreme loss of precision as a deliberate design choice to maintain strict **soundness** (ensuring the analysis is mathematically safe and never underestimates a value). This taught us a fundamental trade-off in static analysis: to guarantee termination and safety, you must sacrifice exact precision. A natural future improvement to recover this lost precision would be implementing a **Narrowing Operator** (Δ), which could pull the infinite value back down to its true mathematical limit once the loop finishes.

8.3. Context Sensitivity is Non-Negotiable for Security

When we tested the Taint checkers, we realized how important context is. Imagine a simple utility method like `print(data)`. If we use a **context-insensitive** analysis, the framework merges all the calls to `print()` together. If one safe user calls it, and one malicious user calls it with tainted data, the analyzer mixes everything into a TOP (unknown) state and flags every single `print()` as dangerous. By enabling **Context-Sensitive analysis**, the framework creates a separate, isolated CFG path for every single function call. This prevents the safe data from mixing with the tainted data, allowing the checker to find the real vulnerability without bothering the programmer with false alarms.

8.4. The Branch Refinement Gap

The most important architectural limitation we documented is the absence of the `assumeBinaryExpression` method in our custom domains (the state refinement phase). Currently, our analyzer is somewhat "blind" to `if` statements. If a programmer writes `if (isClean(x))`, they are intentionally sanitizing the data. However, because our domain does not actively "read" and learn from this boolean condition, it does not refine the abstract state. The variable `x` remains TOP even inside the safe branch, leading to "Possible Warnings." This finding highlights that evaluating arithmetic is not enough, **branch refinement is the most critical feature** needed to build a truly precise static analyzer.

8.5. The "Multiply by Zero" Annihilator is Lost in Abstraction

During our tests, we discovered a fascinating singularity regarding how abstract domains handle multiplication by zero. Human math dictates that multiplying any unknown number by zero results in exactly 0. However, abstract domains often miss this algebraic simplification.

In the **Interval domain**, evaluating $TOP * 0$ translates to $[-\infty, +\infty] \cdot [0,0]$. In the rules of abstract limits, infinity times zero is an indeterminate form, which the domain conservatively approximates to TOP . As a result, an operation guaranteed to yield zero is reported as a "Possible" division-by-zero later in the code.

We observed an identical blind spot in the **Taint domain**. If a **TAINTED** variable is multiplied by 0, the mathematical result is 0, meaning the user's malicious input has been destroyed. The result should logically be **CLEAN**. However, because taint analysis tracks data flow rather than algebraic math, it evaluates $TAINTED * CLEAN$ and safely propagates the infection, outputting **TAINTED**.

This conservative behaviour makes perfect sense for objects or strings (where combining malicious and safe data corrupts the whole result), but it is fundamentally wrong for scalar numbers, where zero acts as a strict mathematical annihilator. This proves that standard domains prioritize generic data-flow rules over specific algebraic simplifications.

8.6. Widening Erases Loop Bounds in Overflow Analysis

In an overflow test, we tracked a loop that simply counted to 5000 (while $i < 5000$). We expected the analyzer to easily prove that the variable fits inside a 32-bit integer. Instead, it triggered an overflow warning.

This happens because of the same root cause found in finding 8.4: the lack of a Narrowing operator and branch refinement. The Widening panic button pushes the counter to $+\infty$ to guarantee termination. Because the domain does not "read" the < 5000 rule to pull the value back down (narrowing), the variable stays at infinity forever. This perfectly demonstrates how missing relational logic impacts not just array bounds, but also standard numerical overflow checks.

9. Conclusions and Future Work

This project successfully demonstrated the practical transition of Abstract Interpretation from pure mathematical theory to applied software engineering. By extending the LiSA static analyzer, we engineered custom abstract domains from scratch (ExtendedSign, FloatInterval, SignTaint) and developed semantic checkers capable of detecting critical runtime errors in IMP programs.

Throughout the development process, we confronted the fundamental challenges of static analysis. We experienced firsthand the intrinsic trade-off between mathematical precision and computational cost, and we engineered practical solutions, such as Multi-Threshold Widening operators, to guarantee the termination of fixpoint algorithms without destroying the analysis results. While the current implementation is mathematically sound and highly effective, our empirical testing has highlighted several key areas for future architectural expansion.

9.1. Expanding Relational Analysis (The Pentagon Domain)

During the implementation of the ArrayBoundsPentagonChecker, we encountered the immense analytical power of relational abstract domains. While non-relational domains (like standard Intervals) isolate variables and inevitably lose precision inside loops, relational domains preserve the mathematical connections between memory elements.

Although we successfully utilized the Pentagon Domain to securely evaluate loop boundary conditions ($i < a.len$), fully mastering and integrating strictly relational domains remains a primary objective for future development. The Pentagon domain represents a highly efficient "weakly relational" sweet spot (tracking strict inequalities like $x < y$).

9.2. Optimizing the Cartesian Product

The SignTaint combined domain successfully proved that we can monitor numeric constraints and data trustworthiness simultaneously. While we successfully collapse unreachable states (forcing any pair containing $\perp$ into a canonical $(\perp, \perp)$ form), the architecture for valid states remains a plain Cartesian Product.

Coupled with this architectural upgrade is the critical need to implement the assumeBinaryExpression method across all custom numerical domains. By engineering this state-refinement phase, conditional branch logic (e.g., if $(x > 0 \ \&\& \text{isClean}(x))$) will dynamically filter the abstract state inside the execution block. This active narrowing would drastically reduce the instances where the analyzer conservatively degrades to TOP (T) after a sanitized branch, closing the precision gap we observed during our experimental validation.

9.3. Final Remarks: Bridging Theory and Practice

Ultimately, the development of this project served as a rigorous bridge between abstract mathematical theory and applied software engineering. Building custom domains and semantic checkers from scratch forced us to confront the practical realities of the static analysis lemma: balancing soundness, precision, and computational performance. While modern frameworks like LiSA abstract away much of the underlying infrastructure, manually engineering the lattice topologies, widening operators, and Cartesian products provided an interesting perspective on how commercial static analyzers guarantee software reliability. This project proves that while absolute precision in static analysis is mathematically impossible due to undecidability (Rice's Theorem), well-engineered abstract domains can still provide robust, automated, and mathematically sound guarantees for modern cybersecurity.